



Distributed Systems

Mutual Exclusion



Recap



Huang's Algorithm (Weight throwing)

Termination detection using global snapshot

Mutual exclusion



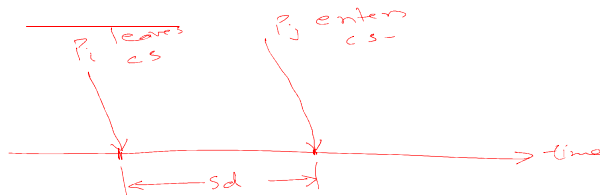
Critical Section Problem

Requirements:

- ✓ Safety
- ✓ Freedom from starvation
- ✓ Freedom from Deadlock
- ✓ Fairness (No global clock)
- ✓ Fault tolerance

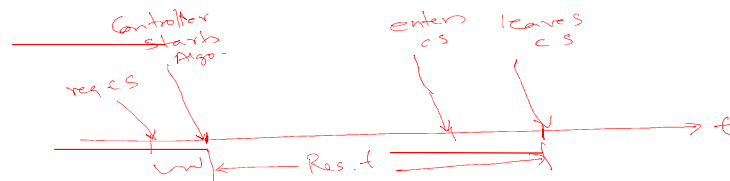
Performance Measures

- Number of messages necessary per CS entry (**message complexity**)
- **Synchronization Delay** (sd): Time required after a site leaves the CS and before the next site enters the CS



Performance Measures

- **Response time:** Time interval a request waits for the CS execution to be over after its request messages have been sent out
- **System Throughput:** Rate at which the system executes requests for CS ($= 1 / (sd + \text{CS exec time } (E))$)



Performance Measures

- Low and High load performance
- Best and worst case performance

Distributed Mutual Exclusion Algorithms

- **Non-token based / Permission based**
 - Permission from all processes: e.g. Lamport, Ricart-Agarwala etc.
- **Token based**
 - ex. Suzuki-Kasami, Raymond's tree based
- **Quorum based** / *Permission based*
 - Permission from a subset: ex. Maekawa
- Tradeoff: Synchronization delay (or throughput) vs message traffic

System Model

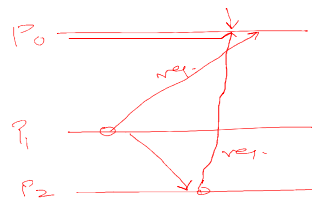
- N sites (processes)
- At any instant, a site may have several requests for CS. A site queues up these requests and serves them one at a time
- A site can be in one of the following states
 - Requesting CS (*blocked*)
 - Executing CS
 - ~~idle~~ *idle* *executing outside CS*
- In the token based algorithms a site can also be in idle token state
- Channels are reliable and there are no faults

Requirements

Algorithm for granting the resources must satisfy following three conditions:

- (I) A process which has been granted the resource must release it before it can be granted to another process.
- (II) Different requests for the resource must be granted **in the order** in which they are made.
- (III) If every process which is granted the resource eventually releases it, then every request is eventually granted.

Centralized Algorithm



- A central scheduling process which grants requests in the order they are received will not work, *for request can't be granted.*
- Let P0 be the scheduling process. Suppose P1 sends a request to P0 and then sends a message to P2. After receiving the latter message, P2 sends a request to P0. It is possible for P2's request to reach P0 before P1's request does.
- Condition II is violated

Lamport's Algorithm

- A site i must acquire permission from **all other sites** to enter into CS
- Every site i has a request queue q_i , keeps requests sorted by logical timestamps
 => total ordering enforced by including process id in the timestamps
- **Messages should be delivered in FIFO order between every pair of sites**

Lamport's Algorithm

PS

To request critical section:

- send timestamped REQUEST (ts_i , i) to all other nodes
- put (ts_i , i) in its own queue

PS

On receiving a request (ts_i , i):

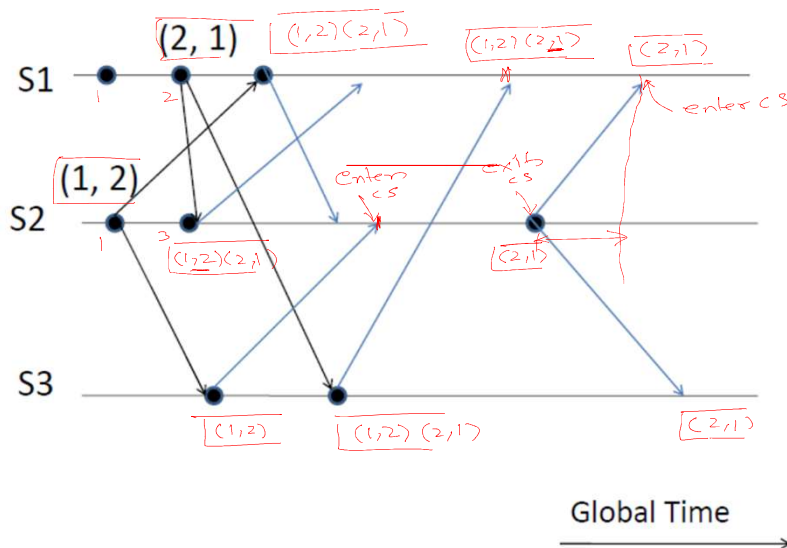
- send timestamped REPLY to the requesting node i
- put request (ts_i , i) in the queue

To enter critical section:

- **L1:** i has received a REPLY message with timestamp larger than (ts_i, i) from ALL other nodes.
- &
- **L2:** i enters critical section if (ts_i, i) is at the top of its own queue, *req are stored in increasing order of ts*

To release critical section:

- i removes its request from its own queue and sends a timestamped RELEASE message to all other nodes
- On receiving a RELEASE message from i , i 's request is removed from the local request queue

Example: Mutual Exclusion

Lamport's Algorithm for Mutual Exclusion



- Lamport's algorithm achieves mutual exclusion

Proof by contradiction:

- Let us assume that two sites S_i and S_j are in CS concurrently and $(ts_i, i) < (ts_j, j)$
 - $\Rightarrow$ For site j to be in CS, both L1 and L2 should be satisfied
 - $\Rightarrow$ ^{L2} j 's request is at the top of its queue &
 - $\Rightarrow$ ^{L1} j has received reply with larger timestamp from all other sites
 - $\Rightarrow$ Reply from i with larger timestamp has been received
 - $\Rightarrow$ request of i has been received (FIFO channels)
 - $\Rightarrow$ request of i should be at the top of Q (contradiction)

IIT ROORKEE

15

Lamport's Algorithm for Mutual Exclusion



- Lamport's algorithm is fair

$\Rightarrow$ Requests are executed in the order of timestamp

Contradiction: Site i & j have requested for CS, site j is in CS and $(ts_i, i) < (ts_j, j)$

- $\Rightarrow$ For site j to be in CS conditions both L1 and L2 should be satisfied
 - $\Rightarrow$ j 's request is at the top of its queue &
 - $\Rightarrow$ j has received reply with larger timestamp from all other sites (including i)
- $\Rightarrow$ reply from i with larger timestamp has been received
- $\Rightarrow$ request of i has been received (FIFO channels)
- $\Rightarrow$ request of i should be at the top of Q (contradiction)

IIT ROORKEE

16

Some points to note:

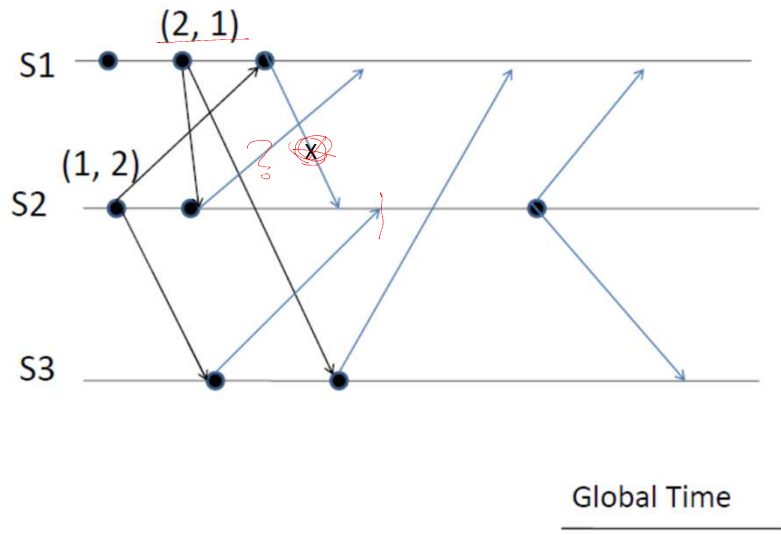
- Requires FIFO channels.
- $3(n - 1)$ messages per critical section invocation
- Synchronization delay = max. message transmission time
- requests are granted in order of increasing timestamps (it is fair)
- It is very unreliable as failure of any one of the processes will halt progress.
- Lamport's algorithm can be optimized to require between $3(n - 1)$ to $2(n - 1)$ messages per CS execution

$(n-1) \text{ req} + (n-1) \text{ reply} + (n-1) \text{ rel}$

Some points to note:

- Purpose of REPLY messages from node i to j is to ensure that j knows of all requests of i prior to sending the REPLY (and therefore, possibly any request of i with timestamp lower than j's request)

Example: Mutual Exclusion



- Lamport's algorithm can be optimized to require between $(3n - 1)$ to $(2n - 1)$ messages per CS execution

To enter critical section:

- **L1:** i has received a message (any message) with timestamp larger than (ts_i, i) from ALL other nodes.
&
- **L2:** i enters critical section if (ts_i, i) is at the top of its own queue,

Ricart-Agarwala Algorithm

- Improvement over Lamport's
- Does not require FIFO channels
- Idea:
 - node j need not send a REPLY to node i if j has a request with timestamp lower than the request of i (since i cannot enter before j anyway in this case)
- requests granted in order of increasing timestamps
- Request-deferred array RD_i of size n where n is the number of processes
- Initially for all i, j: $Rd_i[j] = 0$

To request critical section:

- send timestamped REQUEST message (ts_i, i) to all

On receiving request (ts_j, j) at i:

- send REPLY to j if j is **neither requesting nor executing critical section** or **if j is requesting and i's request timestamp is smaller than j's request timestamp.**
- Otherwise, defer the request and set $RD_j[i] = 1$

To enter critical section:

- i enters critical section on receiving REPLY from all nodes
- $\rightarrow (n-1) \text{ req} + (n-1) \text{ reply}$

To release critical section:

- send REPLY to all deferred requests and set $RD_i[j] = 0$

Example: Mutual Exclusion

